



并行计算

11 分布式系统基础

中国人民大学信息学院 · 鄂金龙



教师简介

• 个人信息

- 手机/微信: 13161433418
- 邮箱: ejinlong@ruc.edu.cn
- 办公室: 信息楼125B

• 研究方向

- 云/边缘计算: 混合云存储、跨云同步、云边协同等
- 智能物联网: 智能物体追踪、无线技术融合等
- 流媒体技术: 多路径直播、短视频传输优化等
- 分布式机器学习: 大数据分析、大模型优化等





本课目录

- 分布式系统概述
- 分布式关键技术
- 专题作业说明





分布式系统概述

• 什么是分布式系统？

- **定义1**：一个分布式系统是一些**独立的计算机的集合**，但是对这个系统的用户来说系统就像**一台计算机一样**
 - 从硬件角度：每台计算机都是自主的
 - 从软件角度：对用户来说就像与单个系统打交道
- **定义2**：一组协同工作的计算机通过**网络互相连接**，互相**传递消息协调**它们的**行为**而形成的系统





分布式系统概述

- **分布式** (Distributed) **vs. 并行** (Parallel)
 - **共同点**
 - 都是**多机处理任务**，重点考虑机器间负载均衡、通信和同步，以处理性能为目标 (*广义上讲分布式包括并行*)
 - **区别**
 - **并行**通常处理一个超大任务，强调并行度，追求计算快，**效率和成本其次**，主要用于**超算** (Super Computing)
 - **分布式**处理多个任务，**综合考虑**性能/**可靠**/**成本**/**服务质量**等目标，企业级实际应用多





分布式系统概述

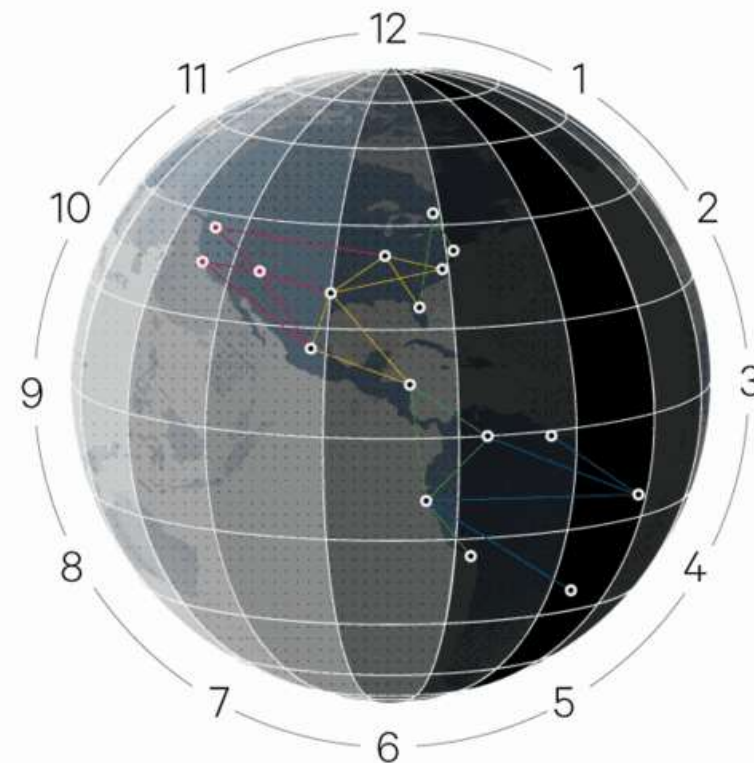
• 分布式系统的优缺点

– 优点

- 微处理机提供比大型主机更好的性能价格比 (经济性)
- 匹配涉及空间分散机器的应用 (固有分布性)
- 单个机器崩溃时系统还可以运转 (可靠性)
- 计算能力可以逐渐有所增加 (可扩展性)

– 缺点

- 目前为分布式系统开发的软件较少 (软件复杂)
- 网络饱和或不稳定会引起系统故障 (通信瓶颈)
- 容易造成对保密数据的访问 (安全漏洞)





分布式系统概述

• 主要挑战

– 并发问题 (Concurrency)

- 分布式计算系统 (无状态) : 任务分配均衡, 数据交互/同步
- 分布式存储系统 (有状态) : 数据分布决定, 多副本有助于负载均衡

– 局部故障 (Partial Failure)

- 规模大出错是常态: 网络数据包丢失、乱序、延迟, 节点随时崩溃、失联
- 要求更高: 需要投入更多资源, 甚至异地容灾 -> 更可靠 & 更可用

– 性能问题 (Performance)

- 性能是永恒的追求: 更偏重扩展性而不是单节点能力, 资源利用率和成本也要考虑
- 系统更为复杂: 会带来性能方面的副作用, 如网络传输、多副本一致性、选举开销



分布式系统概述

- 优化目标

- 性能 (Performance)

- 可扩展性 (Scalability): 有合理的办法应对系统的增长 (数据量、流量、复杂性)
 - 自设计/自调整能力: 面临的访问工作量太复杂, 智能选择模块/方法、参数调优

- 故障容错 (Fault Tolerance)

- 可靠性 (Reliability): 在硬件/软件故障、人为错误时仍能正确完成功能并达到期望性能
 - 可用性 (Availability): 在考察时间内系统能正常运行的概率或时间占有率期望
 - 可恢复性 (Resilience): 整个系统挂掉之后, 再启动还能够恢复到正常的状态继续工作





分布式系统概述

- **优化目标**

- **一致性** (Consistency)

- 分布式共识 (空间): 不同机器的多个副本一致 (Paxos、Raft...)
 - 时间上一致性: 先后到达请求获得的结果一致 (线性一致性、因果一致性、最终一致性)

- **服务质量** (QoS)

- 相比传统的吞吐指标, 更关注每个请求/租户的状态, 一般用尾时延来衡量
 - 服务质量合约 (SLA): 指定服务所提供功能的期望状态的保障协定



分布式系统概述

- 优化目标

- 效率/资源利用率

- 传统模式把资源尽可能多的转为性能，各类资源总有部分浪费，高峰预留利用率很低
 - 云计算模式利用虚拟化技术使底层硬件是可变的，弹性资源按需分配

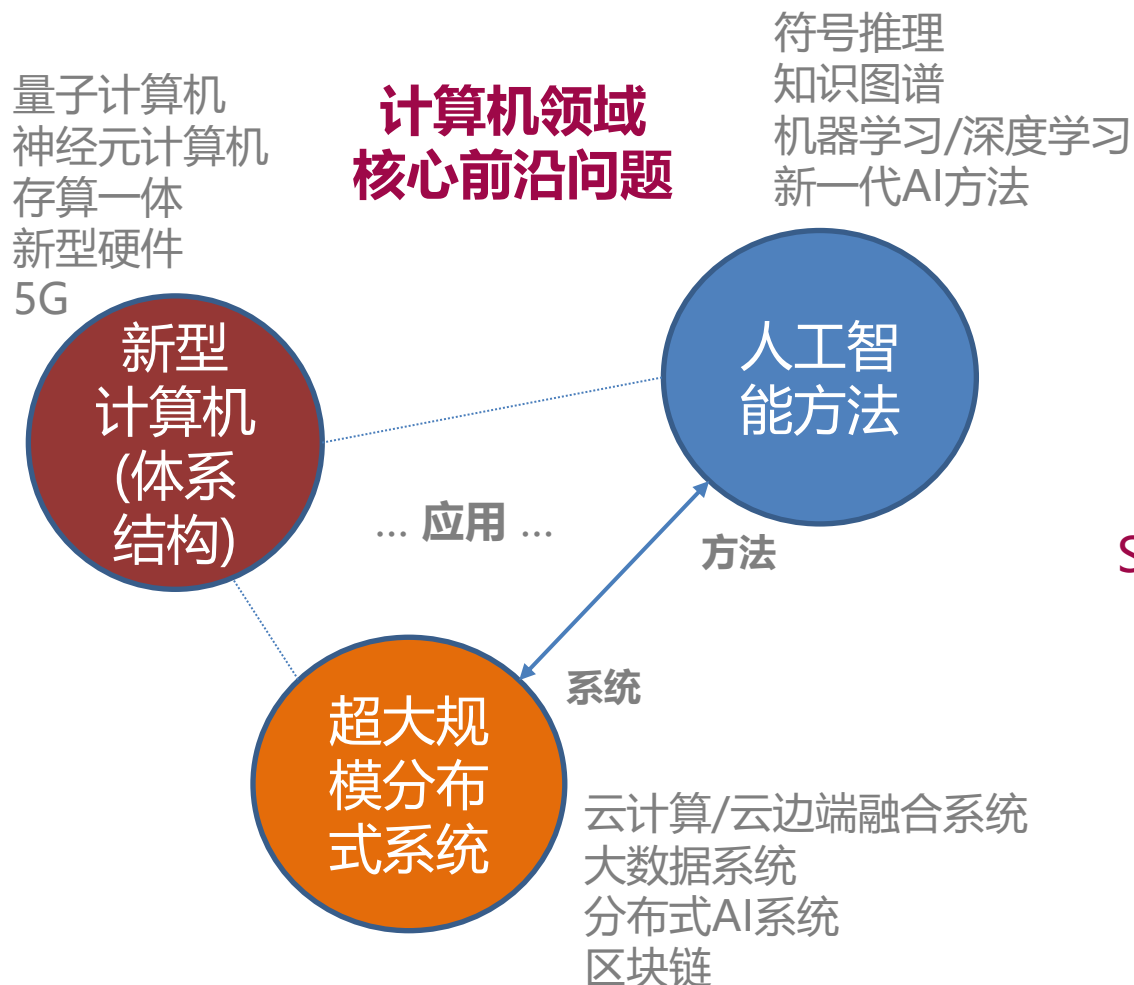
- 可维护性 (Maintainability)

- 可操作性 (Operability): 便于运维团队操作
 - 简单性 (Simplicity): 便于读懂及继承修改代码
 - 可演化性 (Evolvability): 拥抱变化

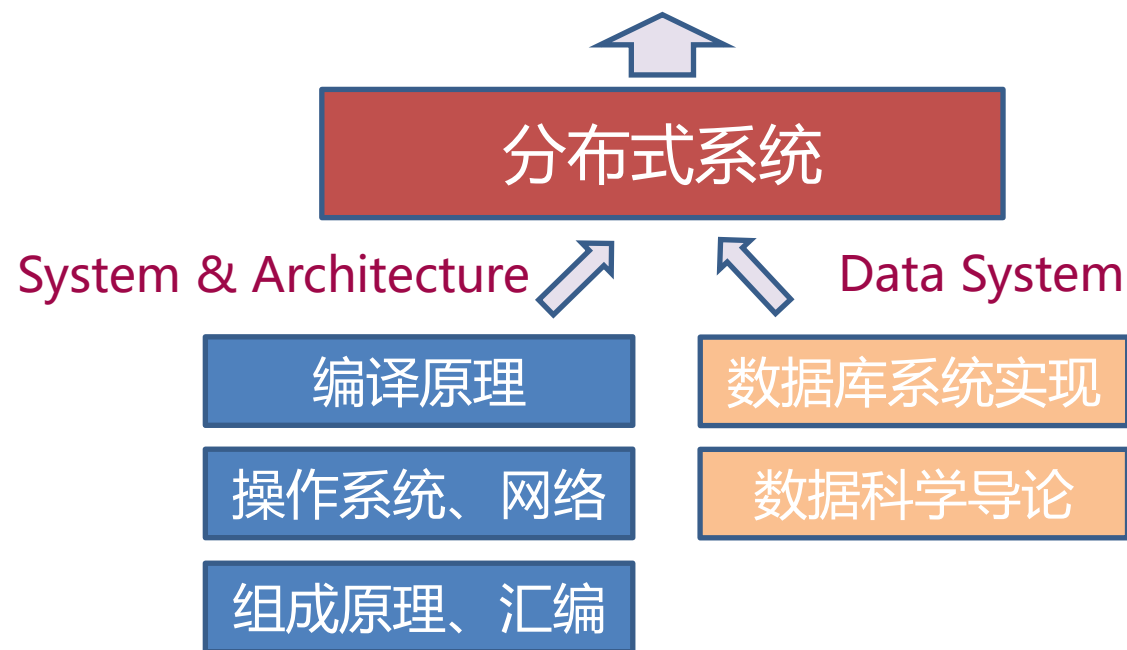


分布式系统概述

• 计算机专业知识体系



分布式文件系统, 分布式数据库, 分布式缓存,
分布式计算系统, 集群管理系统 (Cloud OS),
分布式机器学习, 异构计算平台





分布式关键技术

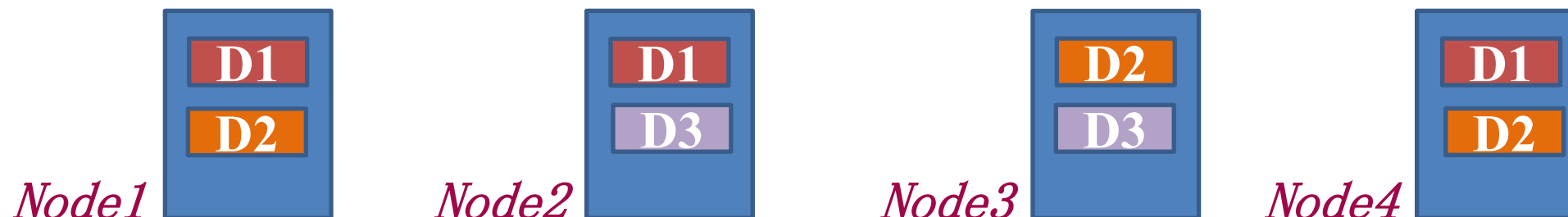
• 分布式存储考虑问题

- 用户如何找到某个数据块的正确位置?
- 如何让各个节点尽可能负载均衡来提升性能?
- 多个副本之间如何保持一致? 写成功 (*Node1*) 后, 如何保障后续读 (*Node4*) 能够读到最新版本?
- 如何管理元数据? 元数据服务器挂了呢?

数据分片 (分区)

数据复制

分布式共识





分布式关键技术

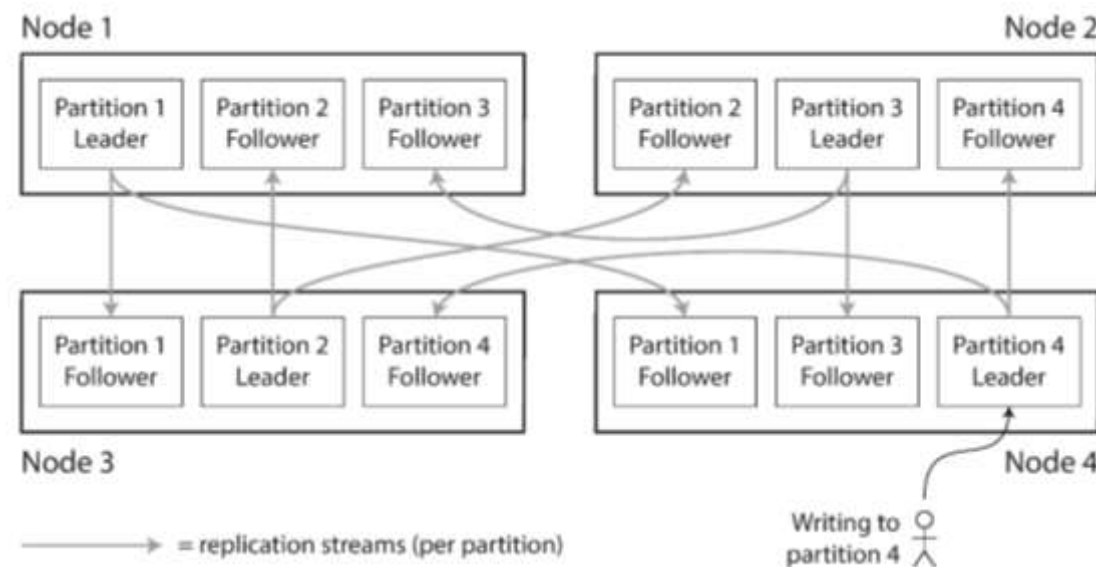
• 数据分片 vs 数据复制

– 区别

- **数据分片**：大数据集**分散**在不同节点上->提高可扩展性 & 增加查询吞吐量
- **数据复制**：在不同节点保存相同数据的**多个副本**->服务高可用 & 防止数据丢失

– 结合

- **主从复制**模型与分片结合使用
- 每个分片的副本存储在多个节点上





分布式关键技术

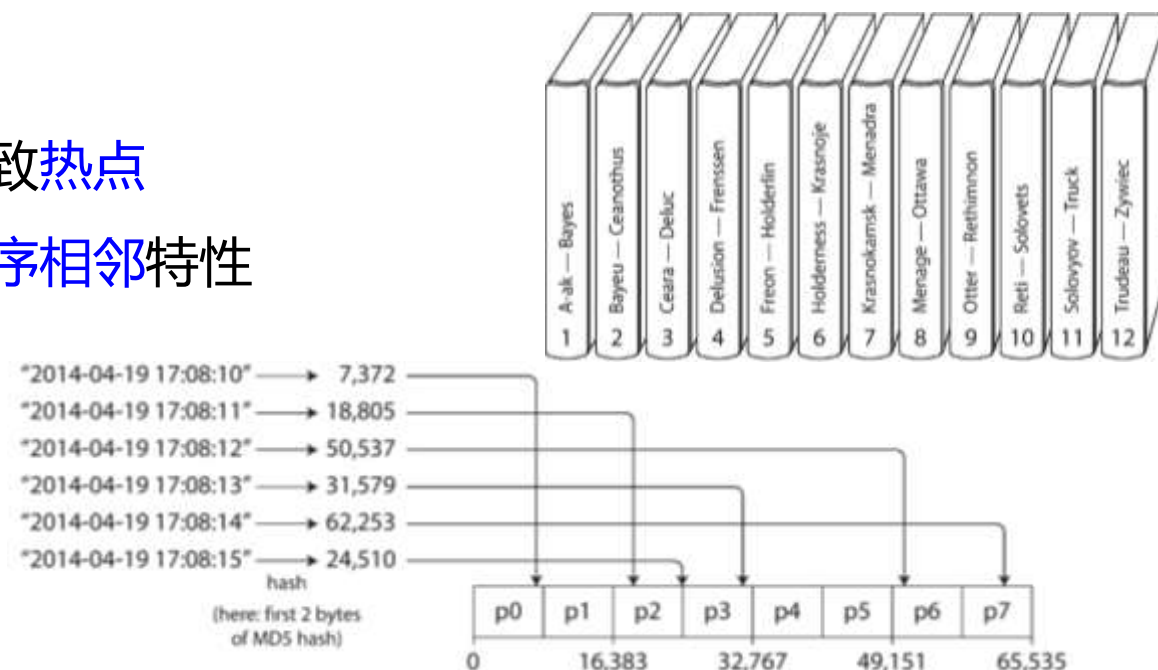
• 数据分片

– 要点

- 对数据集进行**逻辑**划分 -> 将逻辑分片调度到**物理**节点
- 尽量**均匀**，避免**负载偏斜**形成**数据热点**；分片后需要保留**路由信息**

– 分类

- **基于键的范围分片** -> 快速**范围查询**，导致**热点**
- **基于键的散列分片** -> 热点分散，丧失**有序相邻**特性
- **语义分片**：关系数据行/列，图数据点/边





分布式关键技术

• 数据分片

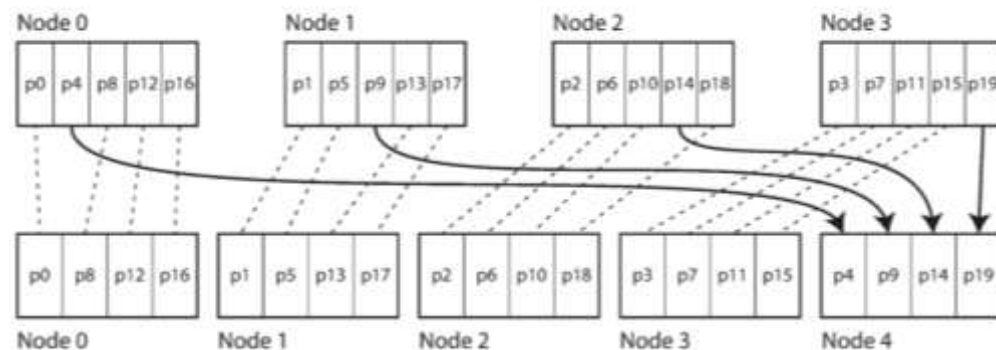
– 分片均衡

- 固定数量的分片：总数固定，节点分配个数不定
- 动态分片：数量适应数据总量，可拆分转移节点
- 按节点比例分片：节点分配个数固定，新增拆分

– 请求路由

- 每个节点有全局路由表：一致性哈希/Gossip
- 有一个专门的路由层：主从架构，协调服务
- 客户端感知分片和节点映射：Memcached

Before rebalancing (4 nodes in cluster)

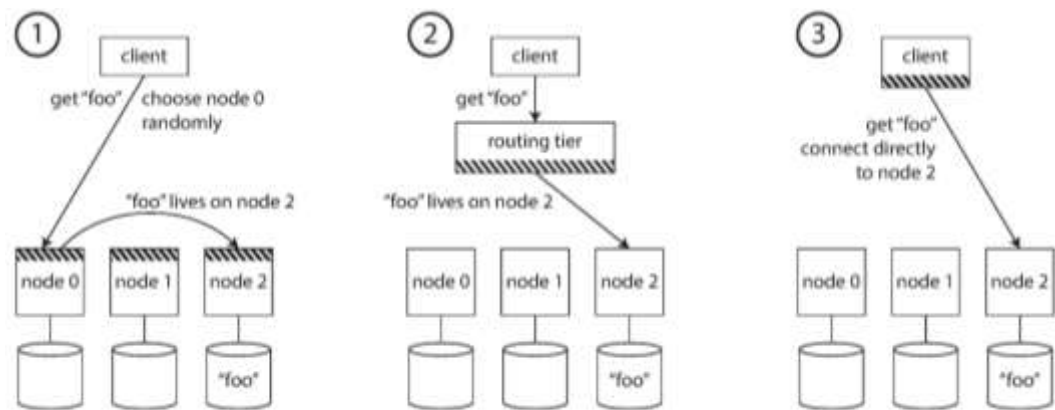


After rebalancing (5 nodes in cluster)

Legend:

----- partition remains on the same node

——> partition migrated to another node



//// = the knowledge of which partition is assigned to which node



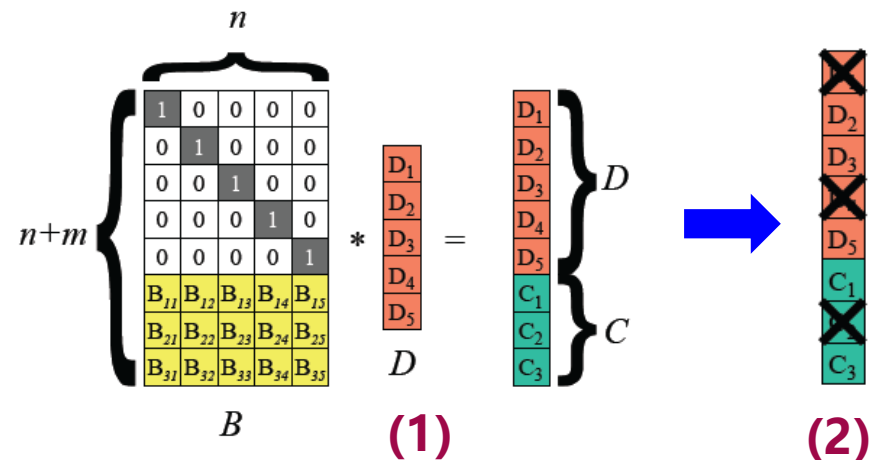


分布式关键技术

• 数据复制

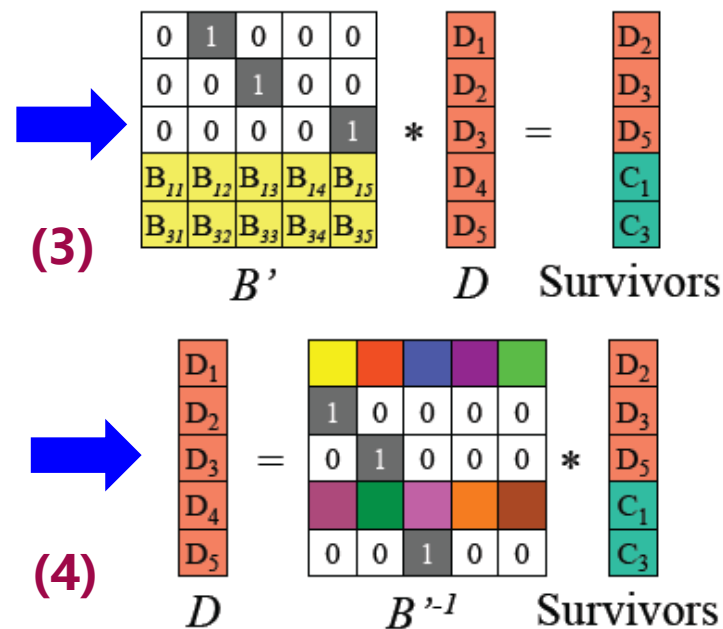
– 多副本 (Multi-Replica)

- 直接复制到多机，适合读写数据，存储量较大
- 分类：主从复制、多主节点复制、无主节点复制



– 纠删码 (Erasure Coding)

- 将n份数据增加m份数据，通过任意n份数据还原原始数据
- 编码以计算换存储，适合只读数据，存储量较小
- 分类：阵列纠删码 (RAID5)、RS纠删码、LDPC纠删码



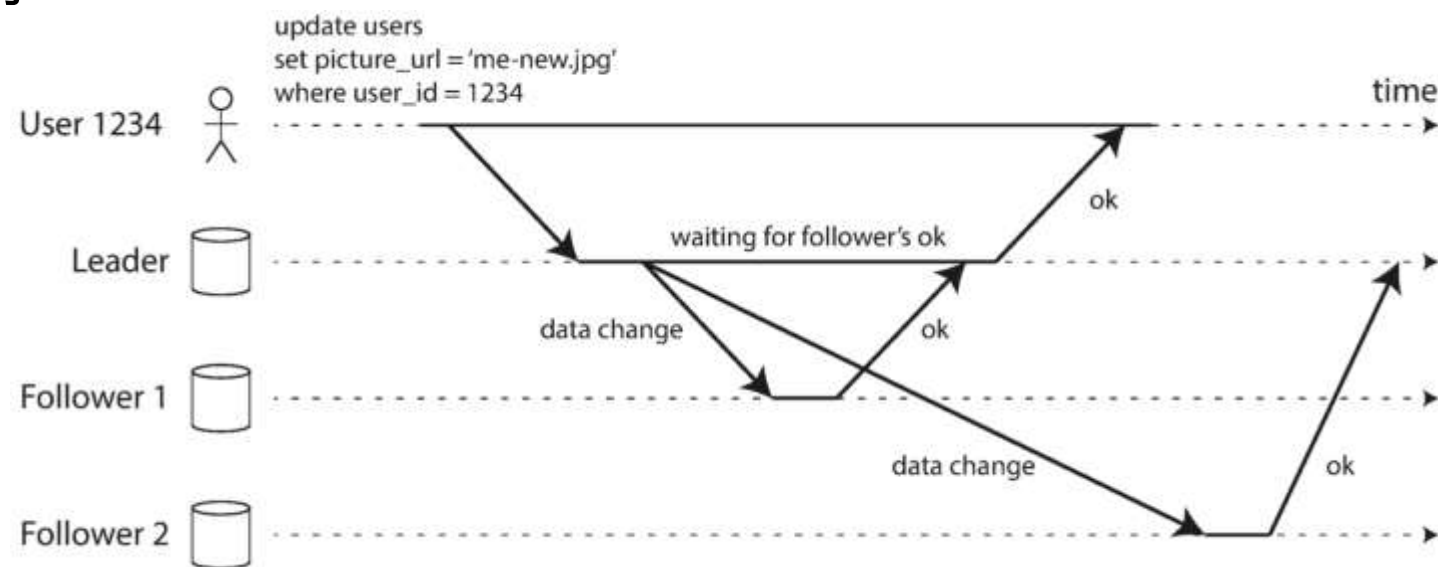


分布式关键技术

• 数据复制

– 主从复制

- 一主多从，从节点可以扩展读性能 -> 主副本/从副本
- 同步复制 vs 异步复制：区别在于请求何时返回客户端，是否等待副本写完？
- 从节点新增：快照+日志复制
- 主节点失效：选举+切换



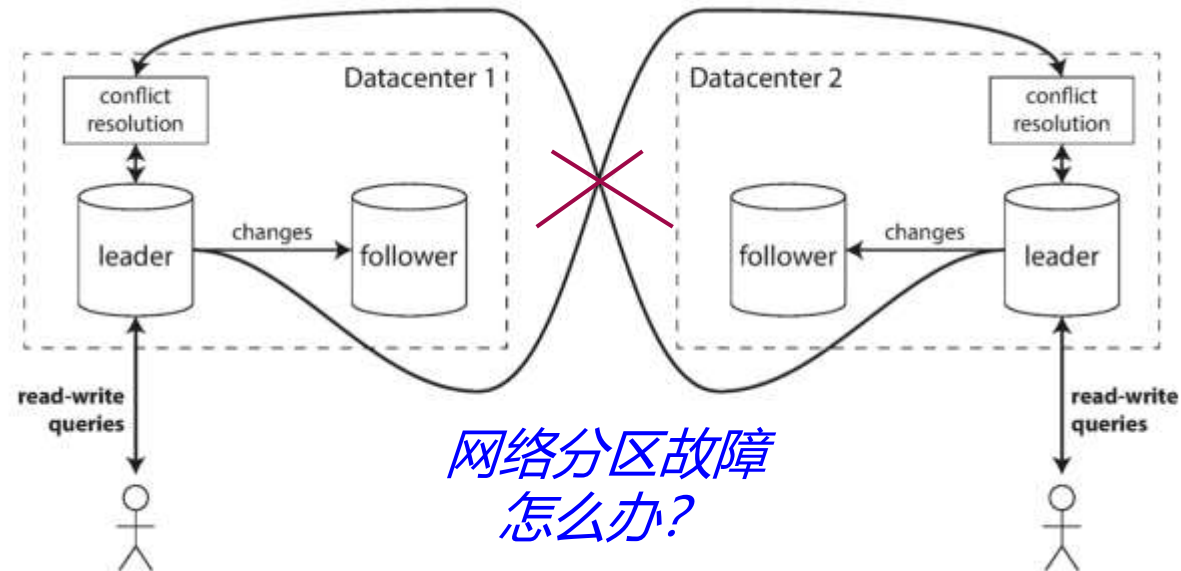


• 数据复制

– 多主节点复制

- 多个可以接受写入的主节点，每个主节点写入后都转给其他所有节点
- 拓扑结构：环型、星型、All-to-All拓扑
- 冲突写处理：合并、高优先级、后者胜、保留冲突

对比项	单主模型	多主模型
性能	所有写入都要路由到一个DC	写入可以就近
可用性	主副本所在DC故障，需要切换	每个数据中心可以独立工作
网络	跨DC写入对网络抖动更敏感	DC间异步复制容错性更高





分布式关键技术

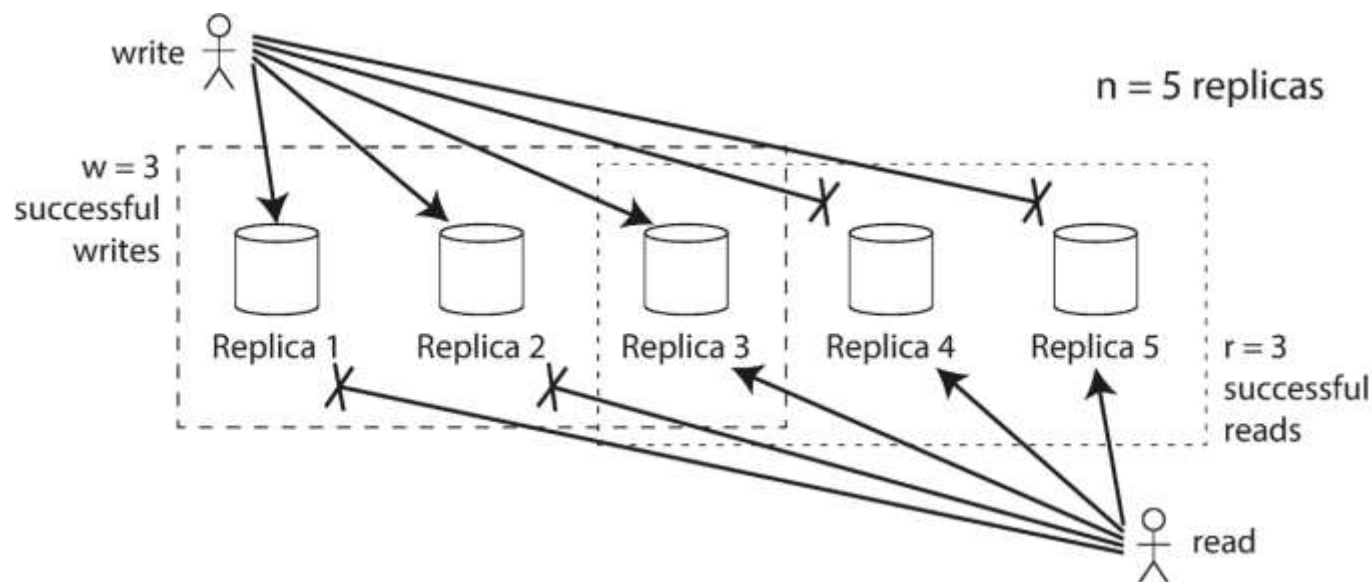
• 数据复制

– 无主节点复制

- 去中心复制，客户端直接发写请求到多副本，读取多个副本选取最新版本值
- Quorum机制：n个副本，写入时需要w个节点确认，读取时至少查询r个节点
-> 只要 $w+r > n$ ，则可保证总能读到最新数据



成功写入的节点集合和读取的节点集合必然有重合





分布式关键技术

• 分布式一致性

– 线性一致性 (Linearizability)

- 提供只有一个副本的假象，并且所有的操作都是原子性的
- 只要成功完成写操作，所有从系统中读取数据必须能够看到刚刚写入的值
- 最高等级一致性，也称原子一致性、强一致性、立即一致性、外部一致性

– 因果一致性 (Casual Consistency)

- 建立因果顺序使得系统可以在消息无序传递的情况下重新构建事件序列
- 填补消息之间的空隙，避免在某些消息仍然缺失的情况下发布操作结果

– 最终一致性 (Eventual Consistency)

- 系统中所有的数据副本在经过一段时间的同步后最终达成一致的状态





分布式关键技术

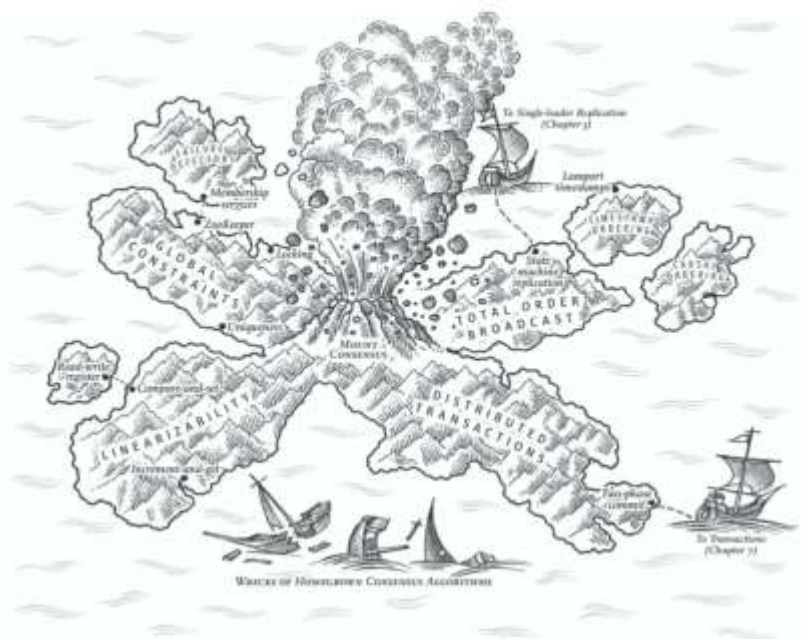
• 分布式共识

– 概念

- **广义**：让所有的节点对某件事达成一致 vs **狭义**：多个副本之间达成一致
- **挑战**：需要全面考虑所有出错情况，如**脑裂**（两个节点都认为自己是领导者）

– Paxos家族：唯一**严格证明**的分布式共识算法/协议

- 基本Paxos算法
- Multi-Paxos算法 (Chubby)
- Raft算法
- ZAB协议 (ZooKeeper)





分布式关键技术

• 分布式共识

– Paxos

- 聚焦于解决**确定一个值**的单一决策问题，需通过组合多个实例实现日志复制
- 理论严谨但实现复杂，需开发者自行处理状态管理，灵活性高但易引发冲突

– Raft

- 面向**日志**复制，将问题分解为**Leader选举**、**日志复制**和**安全性**三个独立模块
- 通过强制角色分离和明确流程（心跳机制）降低实现难度，减少冲突概率

– ZAB

- 能保证**操作顺序性**、面向ZooKeeper基于**主从模式**的**原子广播**协议
- 侧重高吞吐量事务，用于分布式锁等高频读场景，读操作默认**最终一致性**





分布式关键技术

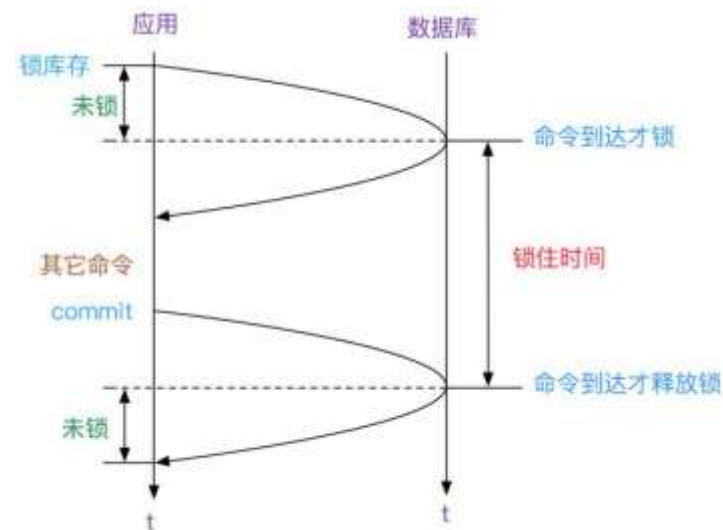
• 分布式事务

– 要点

- 安全性保证：ACID（原子性、一致性、隔离性、持久性）
- 修改多个对象，使用事务ID对操作分组

– 事务隔离

- 读已提交：保证读/写数据库时，不能看到/覆盖未成功提交的数据（脏读/脏写） -> 提交前返回旧版本值+写锁（读倾斜）
- 快照隔离：采用多版本并发控制（MVCC）保留不同版本，读事务只能访问所有版本 $\leq$ txid的数据（写倾斜）
- 可串行化：保证任意时刻只有一个事务在运行
- 两阶段加锁（2PL）：事务执行前为获取锁阶段，提交/中止后为释放锁阶段





分布式关键技术

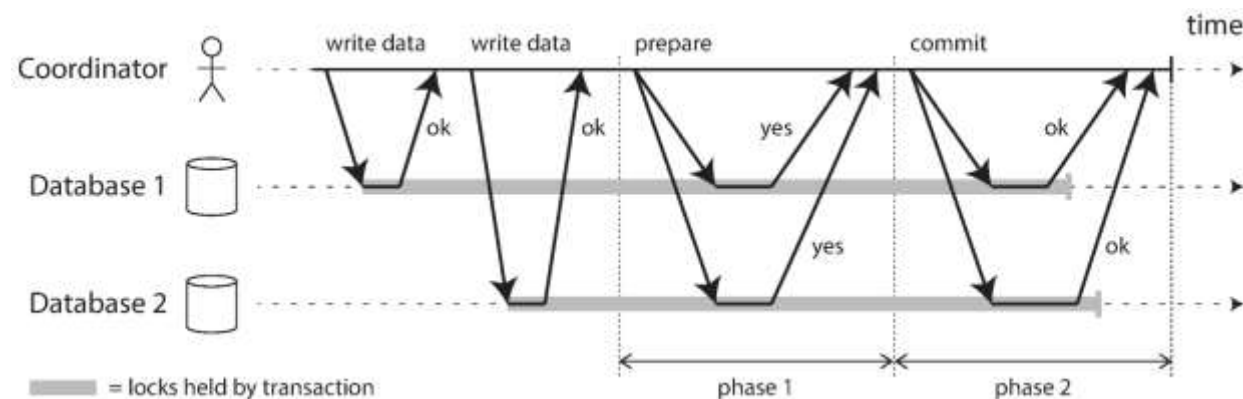
• 分布式事务

– 两阶段提交 (2PC)

- 多节点间阻塞式原子提交协议，确保所有节点要么全部提交，要么全部中止
- 协调者在准备阶段发送准备请求到所有参与者询问是否可以提交并跟踪回复
 - 所有参与者回复“是” -> 协调者在提交阶段发出提交请求，执行提交
 - 有任何参与者回复“否” -> 协调者在提交阶段向所有参与者发出中止请求

– 三阶段提交 (3PC)

- 两阶段提交的准备阶段细分为两个阶段 -> 减少阻塞时间
- 引入超时，参与者等待超时默认提交 -> 解决协调者故障阻塞





分布式关键技术

• 分布式通信

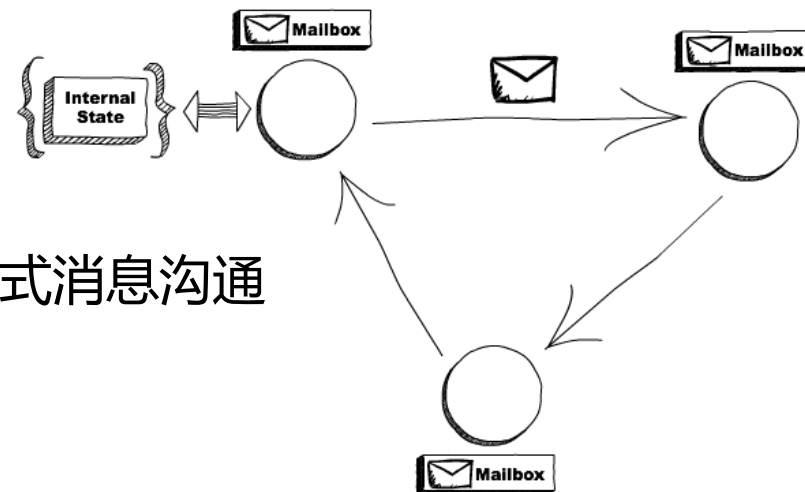
– 基于服务模式 (同步通信)

跨编程语言通信!

- **RPC**: 使向远程网络服务发出请求看起来与本地调用函数相同 (位置透明, 1970s)
- **Web Service**: HTTP作为与服务通信的底层协议 (不限于Web, APP/中间件/OAuth)
- **REST**: 基于HTTP原则的设计理念, 强调简单的数据格式 (符合该原则API称为RESTful)
- **SOAP**: 基于XML的协议WSDL, 带有庞大而复杂的多种相关标准

– 基于消息传递模式 (异步通信)

- **消息代理**: 跨应用/节点传递消息的服务 (隔离应用)
- **分布式Actor框架**: Actor模型接收消息计算, 通过邮件式消息沟通
- 支持消息缓冲、自动重发、多播





分布式关键技术

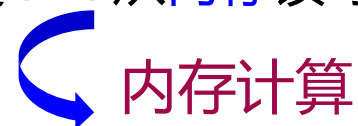
• 分布式计算模式

– 服务 (在线系统)

- 等待客户请求到达，每收到一个立即处理返回响应，**响应时间**和**可用性**是主要指标

– 批处理 (离线系统)

- 大量输入数据通过一个**作业**处理，需要一段时间生成输出数据，通常定期运行，不会有用户等待作业完成，**吞吐量**是主要性能指标 (*MapReduce*)
- 采用列存储、数据分区与压缩、增量写入等优化技术，使CPU从**内存**读写数据 (*Spark*)



内存计算

– 流计算 (准实时系统)

- 介于**在线服务**和**离线批处理**之间，被称为准实时或准在线 (nearline) 处理
- 像批处理一样消费输入并产生输出 (**并不响应请求**)，但在事件发生后不久就进行操作，比批处理具有更低延迟 (*Flink*)





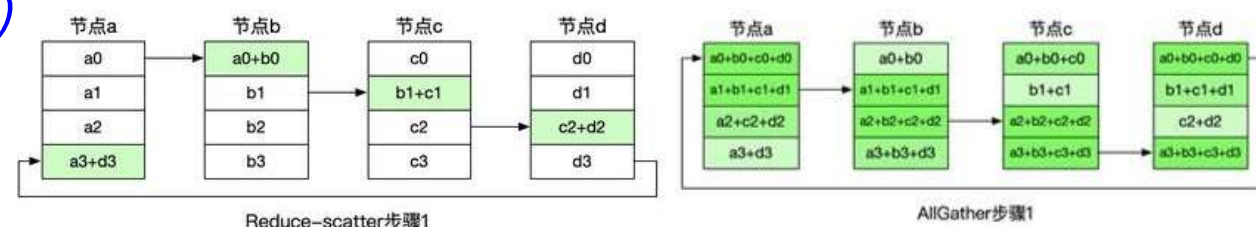
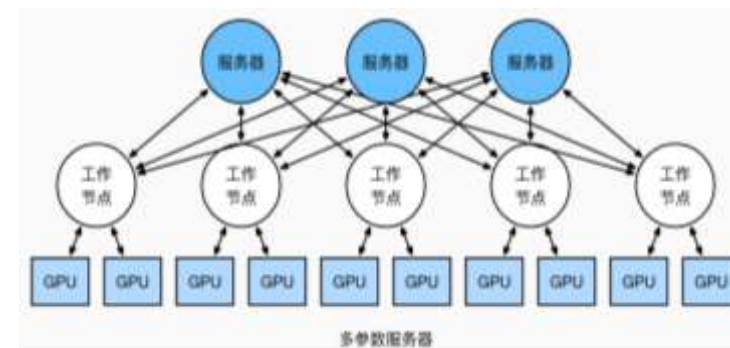
• 分布式机器学习

– 基本思路

- 将训练任务分布式地部署到多个计算节点，提高模型训练速度，减少任务耗时
- 每个计算节点每轮处理样本数量为批尺寸/节点数，各自基于所分配的训练数据对模型进行更新，然后将不同计算节点更新后的模型汇总（每轮结束后在所有节点间同步）

– 参数同步架构

- 参数服务器（PS）架构：参数拉取 -> 本地计算 -> 梯度推送 -> 汇总更新
- 环规约（Ring All-Reduce）架构：本地计算 -> 分散规约 (Scatter reduce) -> 全收集 (All gather) -> 参数更新





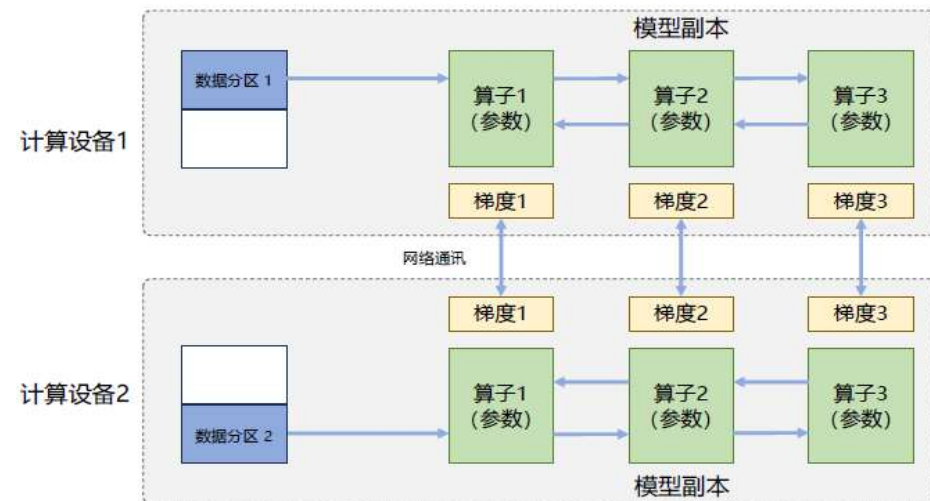
分布式关键技术

• 分布式机器学习

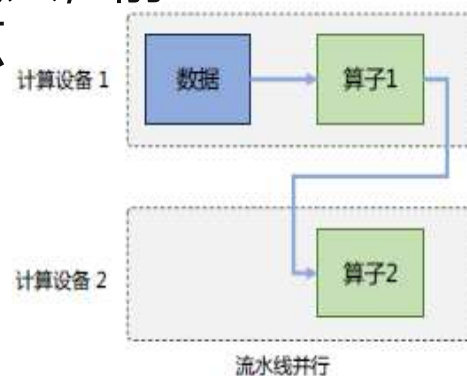
– 并行方式

- **数据并行 (DP)**：每个计算节点均具有**同一模型**的副本，但各自分配的训练**数据不同**，节点间需要将各自的模型更新同步，以保证模型的**全局一致性**
- **模型并行 (MP)**：将模型**划分为多个子模型**，并分别部署在不同计算节点，训练**数据统一输入**，前一节点完成子模型计算将结果传给后一节点

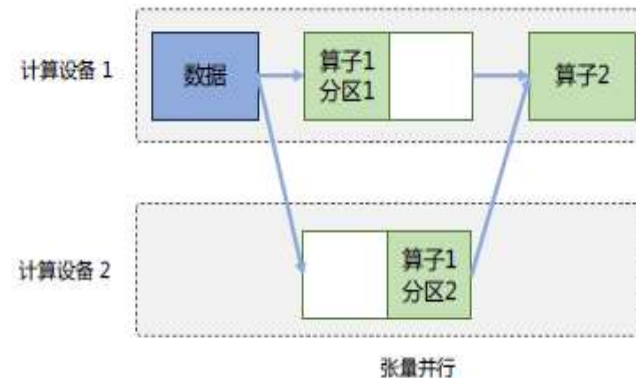
当**参数**数据量较小时选择**数据并行**，
当**激活值**数据量较小时选择**模型并行**



数据并行



流水线并行



张量并行

模型并行

细分为**张量并行 (TP)** & **流水线并行 (PP)**





专题作业说明

- **论文研读报告 (10分)**

- 4个主题：分布式共识、分布式系统、分布式机器学习、云计算
- 根据自己兴趣选择**任一主题两篇以上**推荐论文阅读，撰写一篇**1000字左右**阅读报告
- 重点总结论文的**核心思想**（最好能串联起两篇或多篇论文形成体系），并给出个人**心得体会**（当前研究借鉴之处、改进提升之处等），**结课日（12月23日）提交**

- **论文研讨**

- 按主题分为4个小组，在**第14周（12月9日）**开展研讨，综述主题推荐论文（或其他经典论文）的研究背景和价值、研究内容设计亮点等，**报告15分钟+研讨5分钟**
- 通过填写群问卷报名组队（先到先得），每组最多4人，加入论文报告小组**+2分**，在课堂上积极参与论文研讨**+1分**





专题作业说明

• 分布式共识

- 1. [SIGACT News'01] Leslie Lamport, "Paxos Made Simple" (*Paxos简述*)
- 2. [TOCS'98] Leslie Lamport, "The Part-Time Parliament" (*Paxos*)
- 3. [ATC'14] Diego Ongaro and John Ousterhout, "In Search of an Understandable Consensus Algorithm" (Extended Version) (*Raft*)
- 4. [ATC'10] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed, "ZooKeeper: Wait-free coordination for Internet-scale systems" (*ZooKeeper*)
- 5. [LADIS'08] Benjamin Reed, and Flavio P. Junqueira, "A simple totally ordered broadcast protocol" (*ZAB*)
- 6. [OSDI'06] Mike Burrows, "The Chubby lock service for loosely-coupled distributed systems" (*Chubby*)
- 7. [TPLS '82] Leslie Lamport, Robert Shostak, Marshall Pease. "The Byzantine Generals Problem" (*拜占庭将军问题*)





推荐书目和课程

• 推荐书目

- Martin Kleppmann著, 《数据密集型应用系统设计》, 中国电力出版社, 2018 (*DDIA*, [豆瓣评分9.7](#)) 中文翻译 <https://github.com/Vonng/ddia>
- Dan C. Marinescu著, 《云计算: 原理、应用、管理与安全》, 机械工业出版社, 2021

• 推荐课程

- MIT 6.824 Distributed Systems (Robert Morris) [B站视频](#) [主页课件](#)
- CMU CS15-319: Cloud Computing (Majd F. Sakr) [主页课件](#)





谢谢

